Universitas Esa Unggul
Powered by Arizona State University®

CSF308

OBJECT ORIENTED PROGRAMMING

Sesi 8-9

Inheritance (Basic & Case Study)





Dosen Koordinator:

Dewi Setiowati, A.Md., S.Pd., M.Tr.Kom.

NID:

8413

E-mail:

dewi.setiowati@esaunggul.ac.id

VISI

Menjadi perguruan tinggi kelas dunia berbasis intelektualitas, kreatifitas dan kewirausahaan, yang unggul dalam mutu pengelolaan dan hasil pelaksanaan Tridarma Perguruan Tinggi.

MISI

1. Menyelenggarakan pendidikan tinggi yang bermutu dan relevan
2. Menciptakan suasana akademik yang kondusif
3. Memberikan pelayanan prima kepada seluruh pemangku kepentingan

College Contract

1. Students are required to attend or join the class on time with a maximum **delay of 15 minutes**.
2. Every student's class is required to do a presence **scan QR Code on SIAKAD**
3. Students should attend at least 80% of the scheduled lectures to be able to take the Final test.

Assessment

Coursework evaluation will be weighted as follows:

➢ Mid-Term Examination	10%
➢ Final Examination	10 %
➢ Final Project	50%
➢ Others (class participation 10%, quiz 10%, practical assignments 10%)	30 %
Total	100%

Topik Sebelum UTS

Sesi 1

Introduction to Object Oriented Programming

Sesi 2

Data Type, Variable, Method and Expression

Sesi 3

Control Statement

Sesi 4

Function and Parameter

Sesi 5

Encapsulation (Concept)

Sesi 6

Implementation Encapsulations In Case Study

Sesi 7

Memory Management

Ujian Tengah Semester

Topik Sebelum UAS

Sesi 8

Inheritance Basic

Sesi 9

Inheritance Details (Case Study)

Sesi 10

Polymorphism

Sesi 11

Exception Handling

Sesi 12

Standard Template Library

Sesi 13

Class Presentation

Sesi 14

Class Presentation and Cource Review

Ujian Akhir Semester

Capaian Pembelajaran

Setelah mempelajari materi pada pertemuan ini, diharapkan mahasiswa :

1. Mampu memahami *Inheritance Basic* dan *Inheritance in Case Study*
2. Mampu membuat dan mengimplementasikan *Inheritance* ke dalam sebuah projek dalam Pemrograman Java (NetBeans)

Pendahuluan

Inheritance adalah Salah satu fitur utama OOP yang memungkinkan pengembang membuat kelas baru dari kelas yang sudah ada.

Kelas baru yang dibuat disebut **subkelas** (kelas anak atau kelas turunan) dan **kelas yang sudah ada yang menjadi asal kelas anak** tersebut disebut **superkelas** (kelas induk atau kelas dasar).



Inheritance Basic

INHERITANCE

Inheritance adalah mekanisme di Java yang memungkinkan satu kelas mewarisi fitur (bidang dan metode) dari kelas lain.

Di Java, Inheritance berarti membuat kelas baru berdasarkan kelas yang sudah ada.

Kelas yang mewarisi dari kelas lain dapat menggunakan kembali metode dan bidang kelas tersebut. Selain itu, juga dapat menambahkan bidang dan metode baru ke kelas tersebut.

Mengapa membutuhkan Inheritance di Java?

Kegunaan Ulang Kode: Kode yang ditulis dalam Superclass bersifat umum untuk semua subclass. Kelas anak dapat langsung menggunakan kode kelas induk.

Penggantian Metode: Penggantian Metode hanya dapat dilakukan melalui Inheritance. Ini adalah salah satu cara Java mencapai **Polimorfisme** Waktu Proses.

Abstraksi: Konsep abstrak di mana kita tidak perlu memberikan semua detail, dicapai melalui pewarisan. Abstraksi hanya menunjukkan fungsionalitas kepada pengguna.

Istilah Penting Inheritance di Java?

Kelas adalah sekumpulan objek yang memiliki karakteristik/perilaku dan properti/atribut yang sama. Kelas bukanlah entitas dunia nyata. Kelas hanyalah template atau cetak biru atau prototipe tempat objek dibuat.

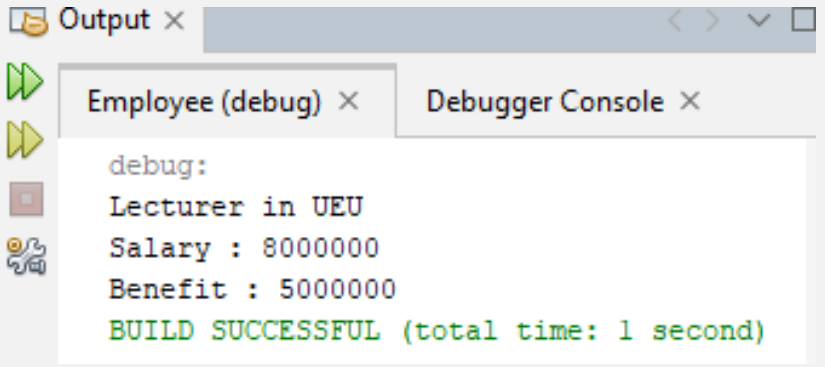
Kelas Super/Kelas Induk: Kelas yang fiturnya diwarisi dikenal sebagai kelas super (atau kelas dasar atau kelas induk).

Subkelas/Kelas Anak: Kelas yang mewarisi kelas lain dikenal sebagai subkelas (atau kelas turunan, kelas lanjutan, atau kelas anak). Subkelas dapat menambahkan bidang dan metodenya sendiri selain bidang dan metode dari superkelas.

INHERITANCE (in case study)

Contoh program : Employee

```
5   package employee;
6
7   /**
8    *
9    * @author TOSHIBA
10   */
11  //Super Class
12  class Employee {
13      int salary = 8000000;
14  }
15  // Inherited or Sub Class
16  class Lecturer extends Employee {
17      int benefits = 5000000;
18  }
19
20  // Main Class
21  class main {
22      public static void main(String args[])
23      {
24          Lecturer El = new Lecturer ();
25          System.out.println("Lecturer in UEU");
26          System.out.println("Salary : " + El.salary + "\nBenefit : "+El.benefits);
27      }
28  }
```

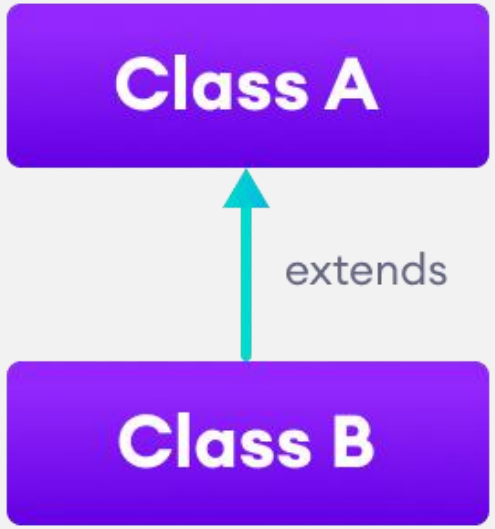




Inheritance Detail

Tipe Inheritance di Java

1. **Single Inheritance**, satu subkelas diturunkan dari satu superkelas.



SINGLE INHERITANCE

```java
class One {
    public void print_pbo()
    {
        System.out.println("Pemrograman Beorientasi Objek");
    }
}
// kelas dua untuk memperluas kelas satu
class Two extends One {
    public void print_for() { System.out.println("dengan JAVA"); }
}

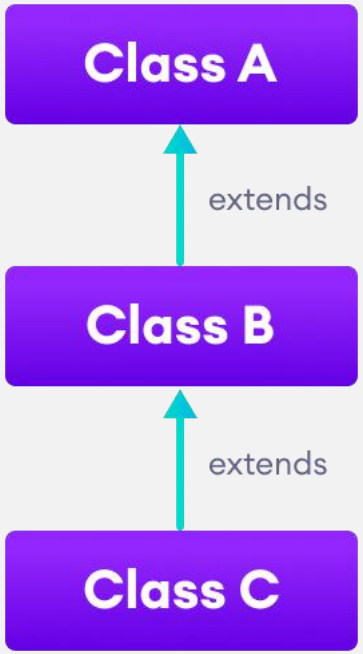
// Main class
class main {
    // Main function
    public static void main(String[] args)
    {
        Two p = new Two();
        p.print_pbo();
        p.print_for();
        p.print_pbo();
    }
}
```



```
debug:
Pemrograman Beorientasi Objek
dengan JAVA
Pemrograman Beorientasi Objek
BUILD SUCCESSFUL (total time: 0 seconds)
```

Tipe Inheritance di Java

2. Multilevel Inheritance, Subkelas diturunkan dari superkelas dan kemudian subkelas yang sama bertindak sebagai superkelas untuk kelas lain.



MULTILEVEL INHERITANCE

```java
class Satu {
    // Method to print "Pemrograman Berorientasi Objek"
    public void print_pbo() {
        System.out.println("Pemrograman Berorientasi Objek");
    }
}
// Child class Dua inherits from class Satu
class Dua extends Satu {
    // Method to print "for"
    public void print_for() {
        System.out.println("dengan JAVA");
    }
}

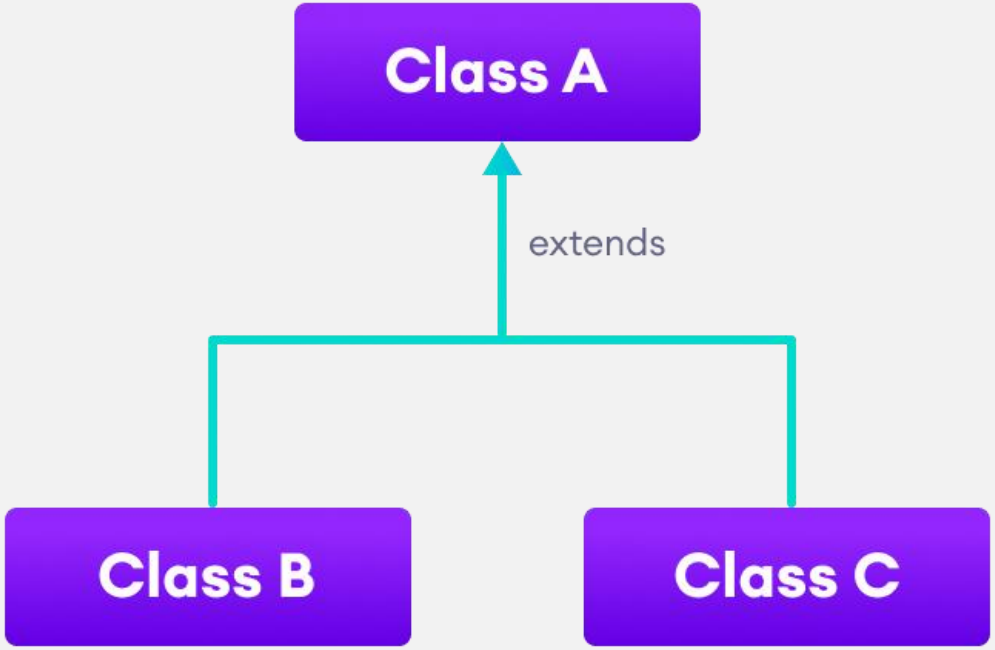
// Child class Tiga inherits from class Dua
class Tiga extends Dua {
    // Method to print "Pemrograman Berorientasi Objek"
    public void print_lastgeek() {
        System.out.println("Pemrograman Berorientasi Objek");
    }
}
```



```
debug:
Pemrograman Berorientasi Objek
dengan JAVA
Pemrograman Berorientasi Objek
BUILD SUCCESSFUL (total time: 0 seconds)
```

Tipe Inheritance di Java

3. Hierarchical Inheritance, beberapa subkelas diturunkan dari satu superkelas.



HIERARCHICAL INHERITANCE

```
class PBO {
    public void print_PBO()
    { System.out.println("Class PBO"); }
}

class ControlStatement extends PBO {
    public void print_ControlStatement()
    { System.out.println("Class ControlStatement "); }
}

class Encapsulation extends PBO {
    public void print_Encapsulation()
    { System.out.println("Class Encapsulation"); }
}

class Inheritance extends PBO {
    public void print_Inheritance()
    { System.out.println("Class Inheritance"); }
}
```

```
// Main Class
class main {
    public static void main(String[] args)
    {
        ControlStatement obj_ControlStatement = new ControlStatement();
        obj_ControlStatement.print_PBO();
        obj_ControlStatement.print_ControlStatement();

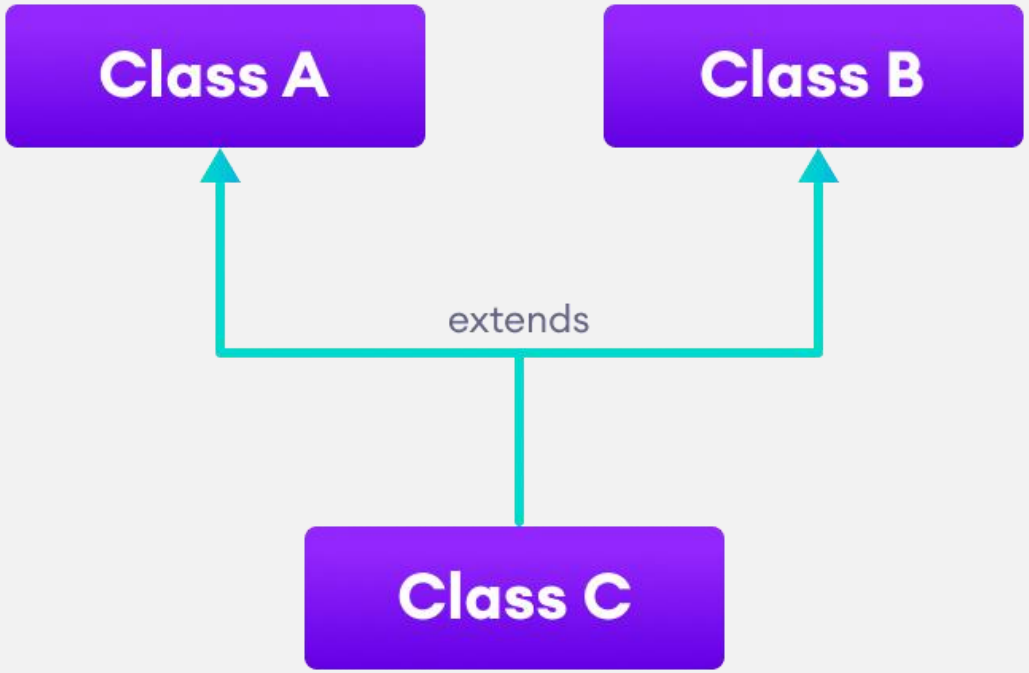
        Encapsulation obj_Encapsulation = new Encapsulation();
        obj_Encapsulation.print_PBO();
        obj_Encapsulation.print_Encapsulation();

        Inheritance obj_Inheritance = new Inheritance();
        obj_Inheritance.print_PBO();
        obj_Inheritance.print_Inheritance();
    }
}
```

```
debug:
Class PBO
Class ControlStatement
Class PBO
Class Encapsulation
Class PBO
Class Inheritance
BUILD SUCCESSFUL (total time: 0 seconds)
```

Tipe Inheritance di Java

4. Multiple Inheritance, Satu subkelas diturunkan dari beberapa superkelas



Note : Java tidak mendukung Multiple Inheritance. Namun, kita dapat mencapai Multiple Inheritance menggunakan interface.

MULTIPLE INHERITANCE

```java
//Multiple harus menggunakan interface
interface Backend {

  // abstract class
  public void connectServer();
}
class Frontend {

  public void responsive(String str) {
    System.out.println(str + " can also be used as frontend.");
  }
}
// Language extends Frontend class
// Language implements Backend interface
class Language extends Frontend implements Backend {

  String language = "Java";

  // implement method of interface
  @Override
  public void connectServer() {
    System.out.println(language + " can be used as backend language.");
  }
```

```java
public static void main(String[] args) {

  // create object of Language class
  Language java = new Language();

  java.connectServer();

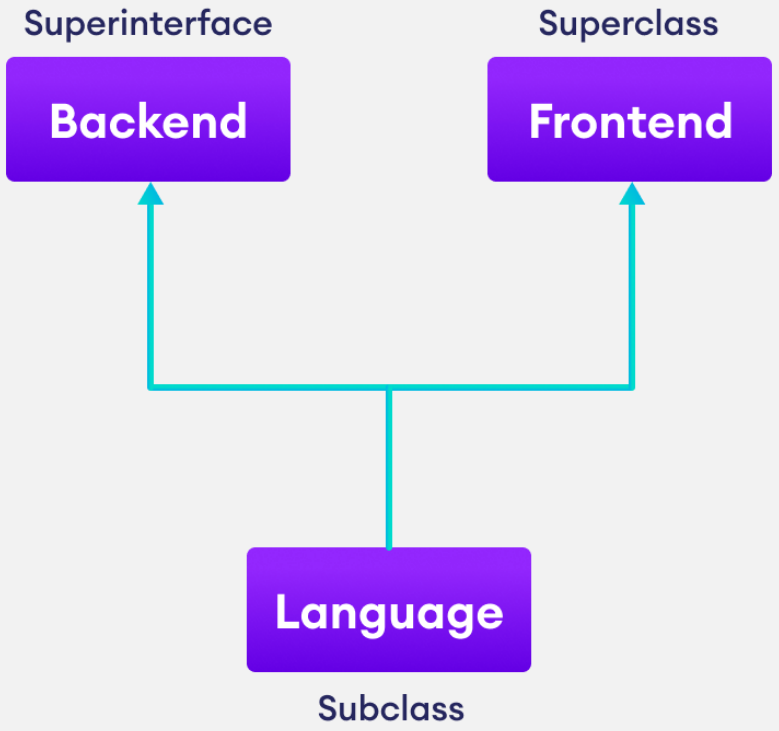
  // call the inherited method of Frontend class
  java.responsive(java.language);
}

}
```

```
debug:
Java can be used as backend language.
Java can also be used as frontend.
BUILD SUCCESSFUL (total time: 0 seconds)
```

metode yang sama hadir di superclass dan subclass, apa yang akan terjadi?
Dalam hal ini, metode di subkelas akan menggantikan metode di superkelas.
Konsep ini dikenal sebagai metode **overriding** -- **connectServer**

MULTIPLE INHERITANCE

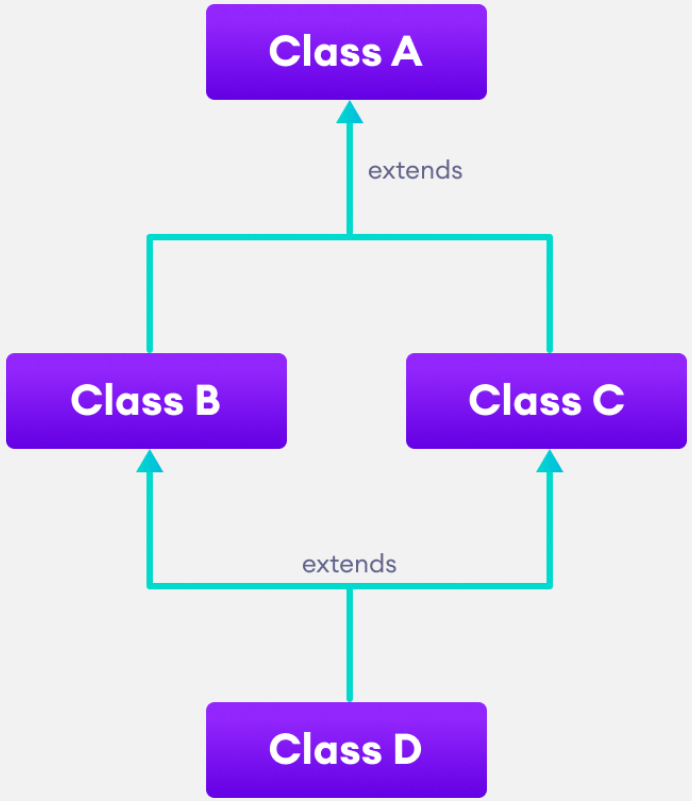


Pada contoh di atas, kita telah membuat **interface** bernama **Backend** dan kelas bernama **Frontend**.
Kelas **Language** memperluas kelas Frontend dan mengimplementasikan interface Backend.

Di sini, kelas Language mewarisi properti Backend dan Frontend. Sehingga dapat disebut dengan Multiple Inheritance (Pewarisan Berganda).

Tipe Inheritance di Java

5. Hybrid Inheritance, Gabungan dari dua jenis inheritance atau lebih.



HYBRID INHERITANCE

```
class Tata_surya {
}
class Bumi extends Tata_surya {
}
class Mars extends Tata_surya {
}
class Bulan extends Bumi {
    public static void main(String args[])
    {
        Tata_surya t = new Tata_surya();
        Bumi b = new Bumi();
        Mars m = new Mars();

        System.out.println(t instanceof Tata_surya);
        System.out.println(b instanceof Bumi);
        System.out.println(m instanceof Tata_surya);
    }

}
```

```
debug:
true
true
true
BUILD SUCCESSFUL (total time: 1 second)
```

Istilah Berorientasi Objek, berikut ini adalah

- Tata Surya merupakan superkelas dari kelas Bumi.
- Tata Surya adalah superkelas dari kelas Mars.
- Bumi dan Mars merupakan subkelas dari kelas Tata Surya.
- Bulan merupakan subkelas dari kelas Bumi dan kelas Tata Surya.

Note : Hybrid Inheritance tidak selalu memerlukan penggunaan Multiple Inheritance secara eksklusif. Ini dapat dicapai melalui kombinasi Multilevel Inheritance dan Hierarchical Inheritance dengan class, Hierarchical Inheritance dan Single Inheritance dengan class. Oleh karena itu, memang mungkin untuk mengimplementasikan Hierarchical Inheritance menggunakan kelas saja, tanpa bergantung pada jenis Multiple Inheritance.

QUIZ

1. Buatlah inheritance mengenai employee (tambahkan class turunan lain selain diatas)
2. Buatlah inheritance mengenai Fruits sebagai super class
3. Buatlah inheritance mengenai Person sebagai super class

TASK (DEMO PROGRAM)

Implementasikan **inheritance** dalam program yang telah Anda buat masing”.

Universitas Esa Unggul
Powered by Arizona State University®

Terima Kasih

